



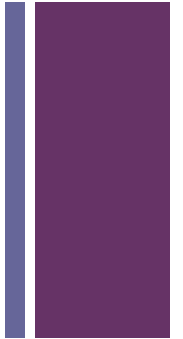
+

Chương 12

Chức năng và cấu trúc Vi xử lý



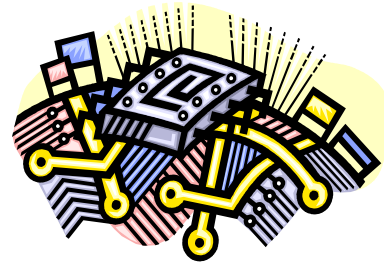
Nội dung



1. Tổ chức của Bộ xử lý
2. Tổ chức thanh ghi
3. Chu kỳ lệnh
4. Kỹ thuật đường ống lệnh (Pipelining)
5. Kiến trúc VXL tiên tiến



12.1 Tổ chức bộ vi xử lý



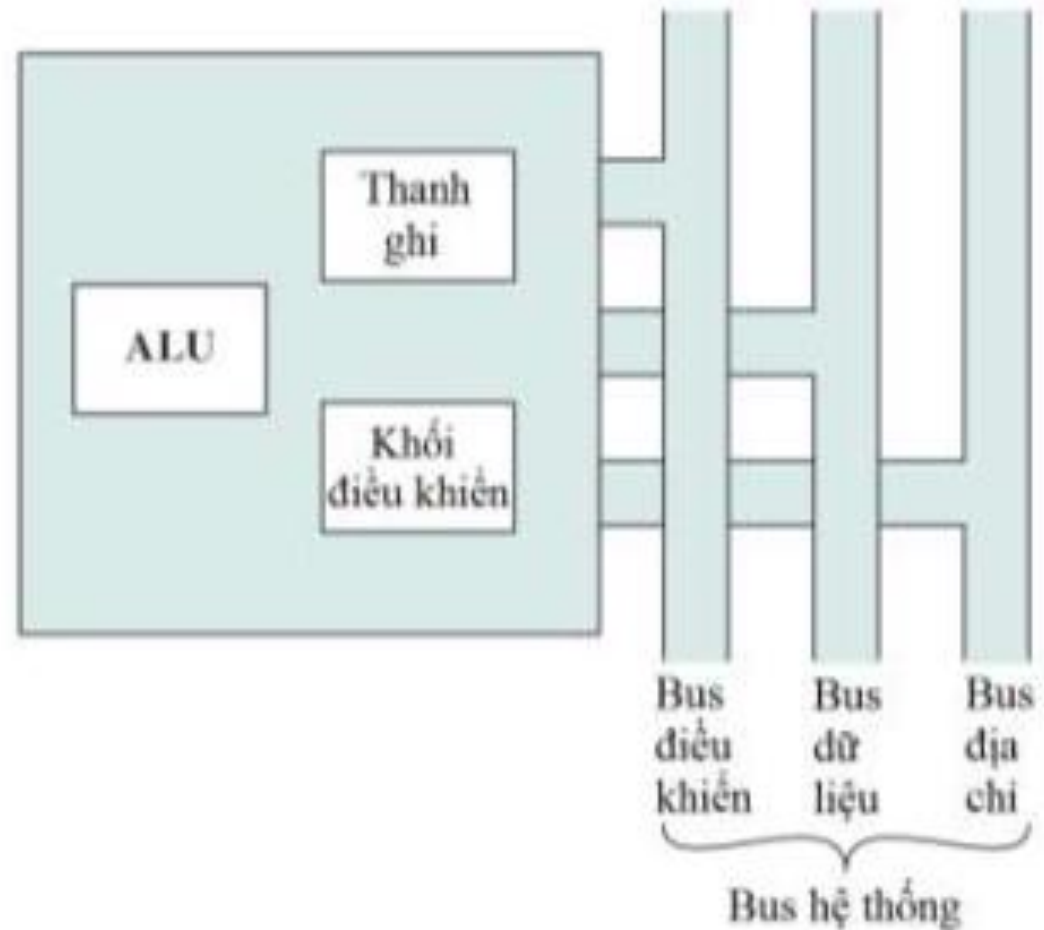
Các yêu cầu xử lý:

- **Truy xuất lệnh:** Bộ xử lý đọc lệnh từ bộ nhớ (thanh ghi, bộ nhớ cache, bộ nhớ chính).
- **Giải mã lệnh:** Lệnh được giải mã để xác định hành động nào được yêu cầu.
- **Truy xuất dữ liệu:** Việc thực thi một lệnh có thể yêu cầu đọc dữ liệu từ bộ nhớ hoặc một module vào/ra
- **Xử lý dữ liệu:** Việc thực thi một lệnh có thể yêu cầu thực hiện một số phép toán số học hoặc logic trên dữ liệu.
- **Ghi dữ liệu:** Kết thúc việc thực hiện có thể yêu cầu ghi dữ liệu vào bộ nhớ hoặc một module vào/ra.

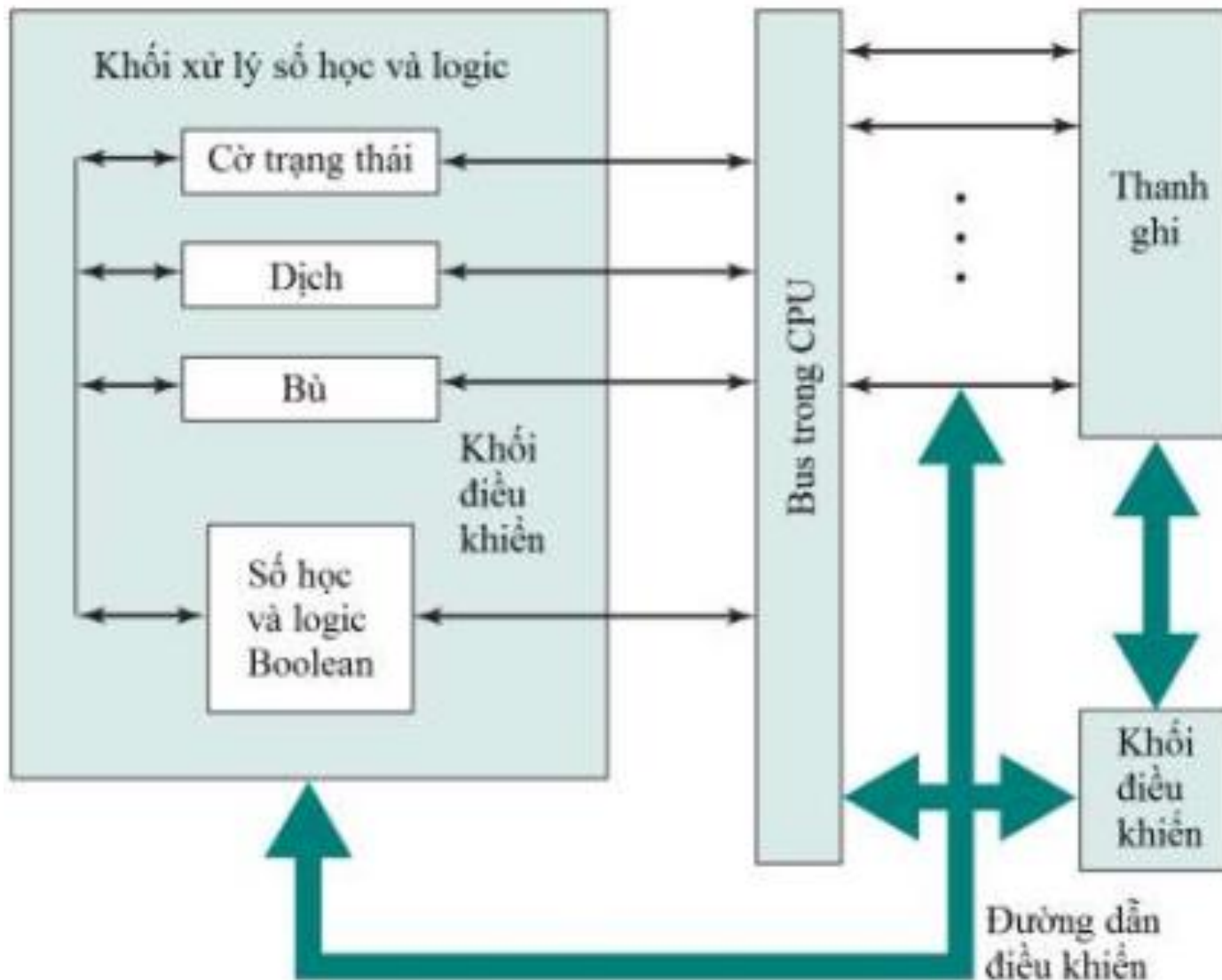
Để thực hiện những việc này, bộ vi xử lý cần lưu tạm thời một số dữ liệu → cần một bộ nhớ nhỏ bên trong → **thanh ghi**

+ Tổ chức VXL

- ALU: khối tính toán số học và logic
- CU: khối điều khiển: kiểm soát việc di chuyển dữ liệu và lệnh vào và ra khỏi bộ xử lý và điều khiển hoạt động của ALU
- Các thanh ghi: lưu trữ dữ liệu tạm thời trong quá trình lệnh được thực hiện



Cấu trúc bên trong CPU



Hình 12.2 Cấu trúc bên trong CPU



12.2 Tổ chức thanh ghi



- Các thanh ghi là một loại bộ nhớ.
- Vai trò của thanh ghi:

Thanh ghi **hiển thị với người dùng**

- Cho phép **người lập trình ngôn ngữ assembly hoặc ngôn ngữ máy** sử dụng trong các câu lệnh
 - VD: ADD AX, 300: lấy dữ liệu ở ngăn nhớ 300 cộng với AX và ghi kết quả vào AX
- Giảm thiểu các tham chiếu bộ nhớ chính bằng cách sử dụng thanh ghi

Thanh ghi điều khiển và trạng thái

- Được sử dụng bởi CU để điều khiển hoạt động của bộ vi xử lý và bởi các chương trình hệ điều hành được đặc quyền (privileged) để kiểm soát việc thực thi chương trình
 - VD: thanh ghi PC chứa địa chỉ lệnh tiếp theo của chương trình

a. Thanh ghi hiển thị với người dùng

- Là các thanh ghi lập trình viên có thể sử dụng trong các lệnh để phục vụ cho mục đích viết chương trình của mình
- Phân loại
 - **Thanh ghi đa năng:** lập trình viên có thể sử dụng các thanh ghi nhóm này cho nhiều mục đích khác nhau
 - **Thanh ghi dữ liệu:** sử dụng để chứa dữ liệu và không dùng để tính toán địa chỉ toán hạng.
 - **Thanh ghi địa chỉ:** có thể là thanh ghi đa năng hoặc là thanh ghi dành riêng cho một chế độ địa chỉ cụ thể.
 - VD: thanh ghi SP (con trỏ đoạn), thanh ghi index, thanh ghi SP (con trỏ ngăn xếp)
 - **Mã điều kiện**
 - Còn gọi là bit cờ
 - Là các bit do phần cứng của bộ xử lý đặt theo kết quả của hoạt động



b. Thanh ghi điều khiển và trạng thái

Bốn thanh ghi cần thiết để thực thi lệnh:

- Thanh ghi PC - Bộ đếm chương trình
 - Chứa địa chỉ của lệnh sắp được truy xuất
- Thanh ghi IR – thanh ghi lệnh
 - Chứa lệnh đang được truy xuất
- Thanh ghi MAR – thanh ghi địa chỉ bộ nhớ
 - Chứa địa chỉ của một vị trí trong bộ nhớ
- Thanh ghi MBR (hoặc MDR) – thanh ghi đệm bộ nhớ
 - Chứa một từ dữ liệu sắp được ghi vào bộ nhớ hoặc từ vừa được đọc ra từ BN
- Một số BXL còn có một hoặc nhiều thanh ghi PSW (program status word - từ trạng thái chương trình): chứa thông tin trạng thái của chương trình đang được thực hiện





b. Thanh ghi điều khiển và trạng thái

Thanh ghi PSW – Thanh ghi trạng thái chương trình

- Thanh ghi hoặc tập hợp thanh ghi chứa thông tin trạng thái và mã điều kiện
- Các trường hoặc cờ phổ biến gồm:
 - Sign: Chứa bit dấu của kết quả của phép tính số học cuối cùng.
 - Zero: Thiết lập khi kết quả bằng 0.
 - Carry: Thiết lập nếu một phép tính có nhớ (phép cộng) hoặc vay (phép trừ) vào bit có bậc lớn hơn. Được sử dụng cho các phép tính số học nhiều từ.
 - Equal: Thiết lập nếu kết quả so sánh logic là bằng nhau.
 - Overflow: Được sử dụng để chỉ định sự tràn số học.
 - Interrupt Enable/Disable: Được sử dụng để cho phép hoặc vô hiệu hoá ngắt.
 - Supervisor: Cho biết bộ xử lý đang thực hiện trong chế độ giám sát hay chế độ người dùng. Một số lệnh privileged chỉ có thể được thực hiện trong chế độ giám sát, và một số vùng bộ nhớ chỉ có thể được truy cập trong chế độ giám sát.

+ c. Ví dụ tổ chức thanh ghi

MC68000, Intel 8086, Intel 80386



Thanh ghi dữ liệu

D0	
D1	
D2	
D3	
D4	
D5	
D6	
D7	

Thanh ghi địa chỉ

A0	
A1	
A2	
A3	
A4	
A5	
A6	
A7	

Trạng thái chương trình

Program counter
Status register

(a) MC68000

Thanh ghi tổng quát

AX	Accumulator
BX	Base
CX	Count
DX	Data

Con trỏ và chỉ số

SP	Stack ptr
BP	Base ptr
SI	Source index
DI	Dest index

Segment

CS	Code
DS	Data
SS	Stack
ES	Extrat

Trạng thái chương trình

Flags
Instr ptr

(b) 8086

Thanh ghi tổng quát

EAX	AX
EBX	BX
ECX	CX
EDX	DX

ESP	SP
EBP	BP
ESI	SI
EDI	DI

Trạng thái chương trình

FLAGS register
Instruction pointer

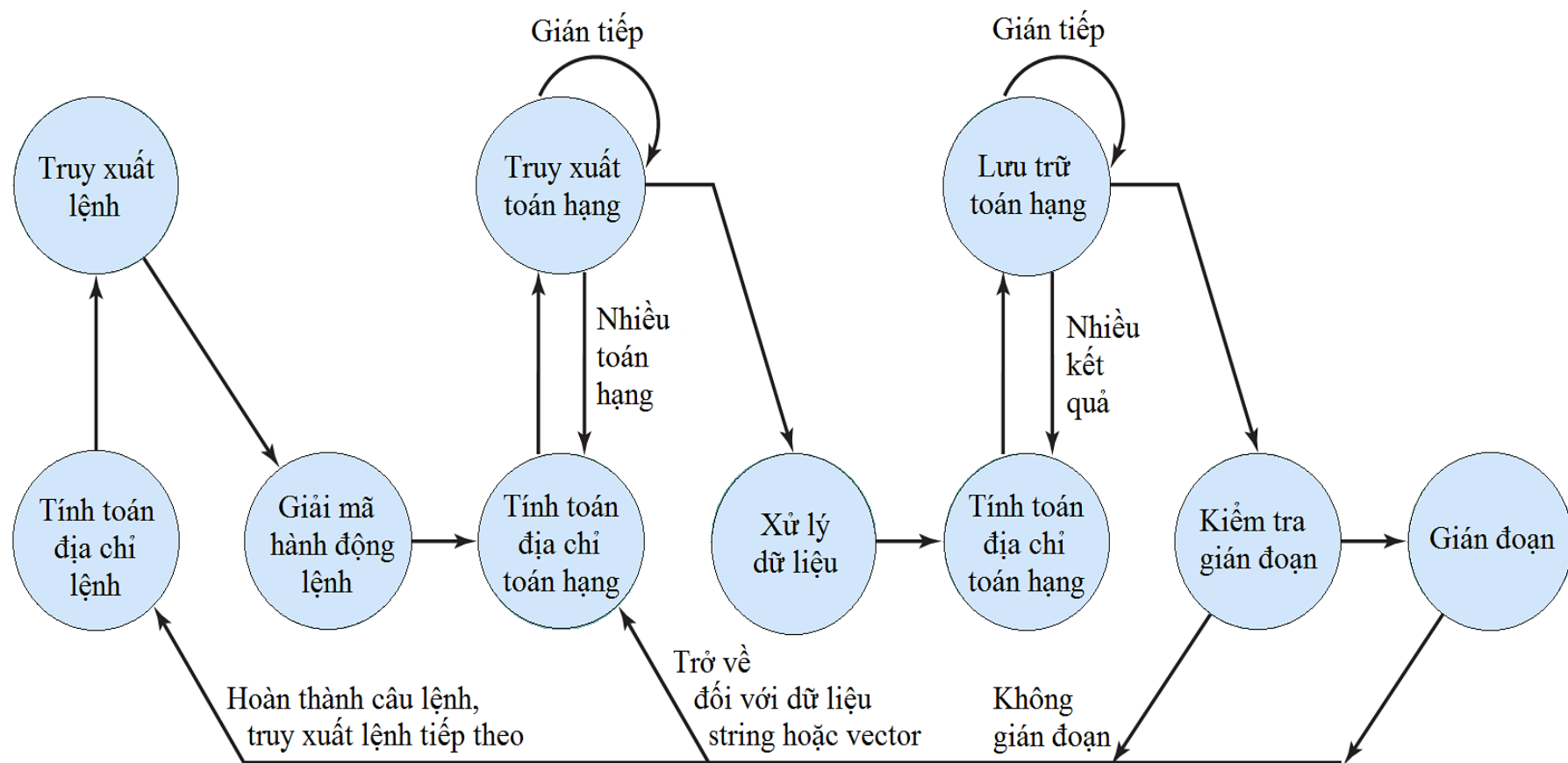
(c) 80386—Pentium 4

12.3 Chu kỳ lệnh

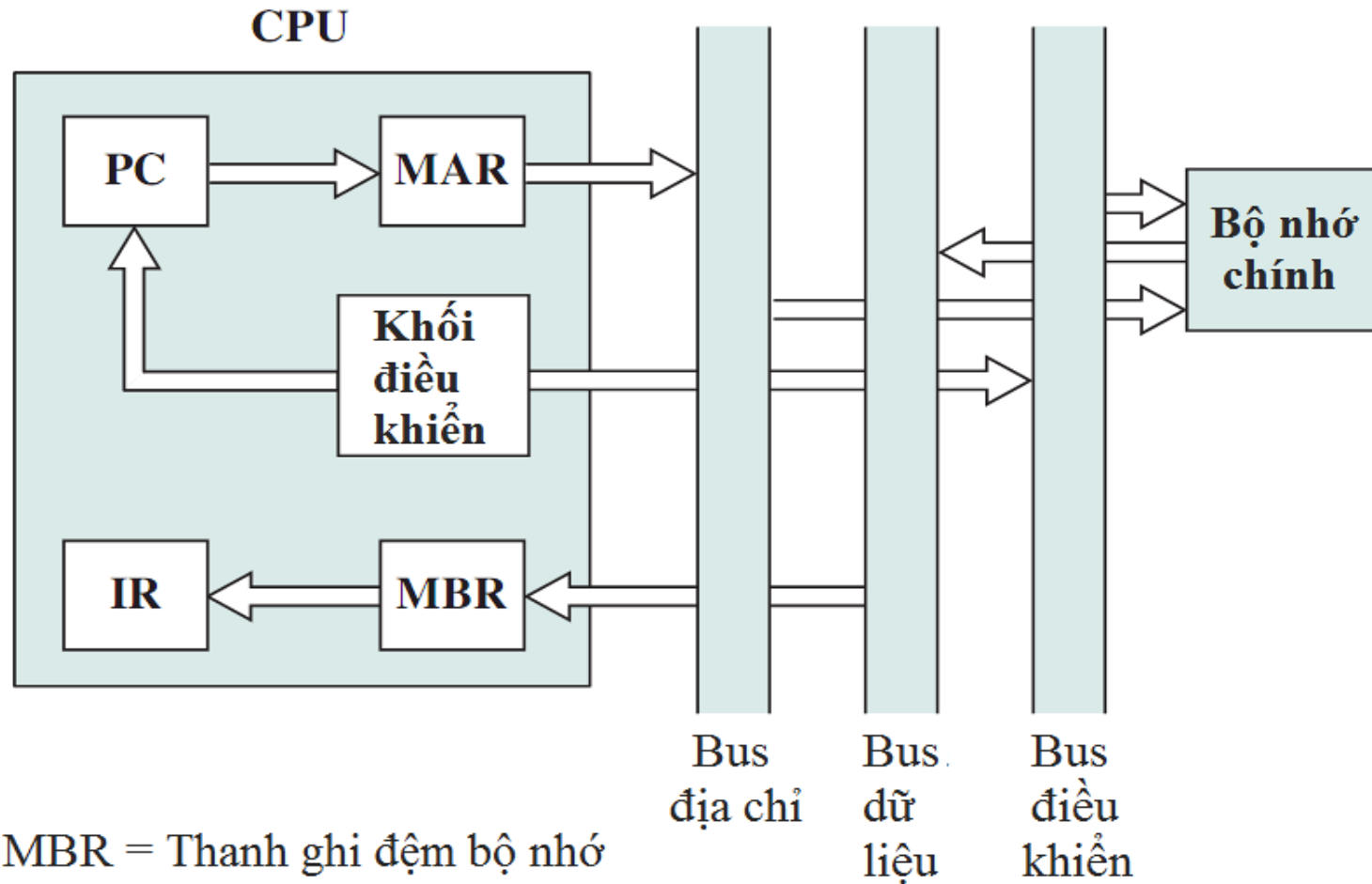
Bao gồm các tầng sau:

- Truy xuất
 - Đọc lệnh tiếp theo từ bộ nhớ vào bộ vi xử lý
- Thực thi
 - Giải mã opcode và thực hiện hoạt động được chỉ định
- Ngắt
 - Nếu có yêu cầu ngắt được gửi đến, VXL lưu trạng thái hiện tại của chương trình và chuyển sang phục vụ ngắt

Sơ đồ trạng thái chu kỳ lệnh



Luồng dữ liệu, chu kỳ truy xuất



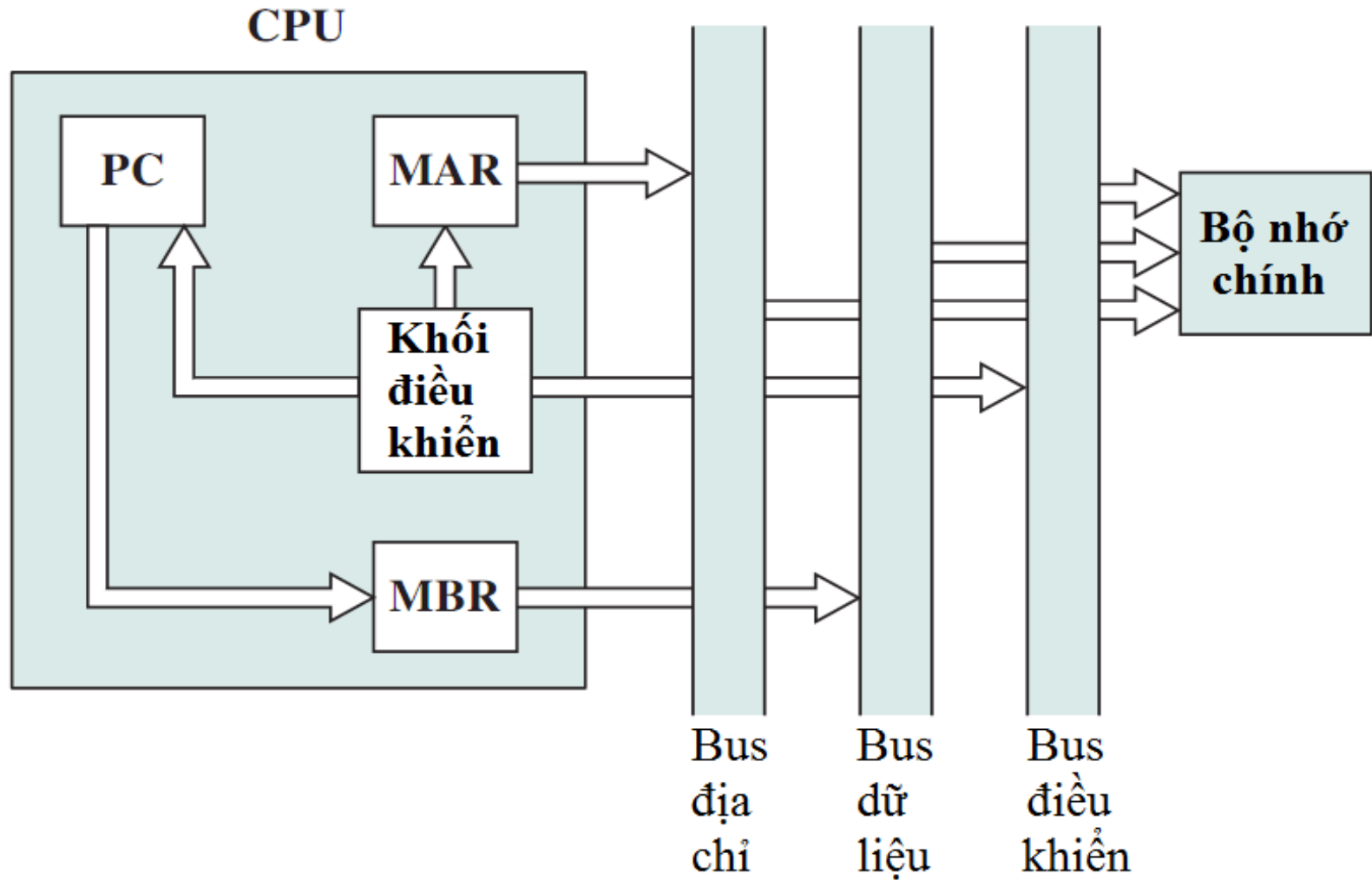
MBR = Thanh ghi đệm bộ nhớ

MAR = Thanh ghi địa chỉ bộ nhớ

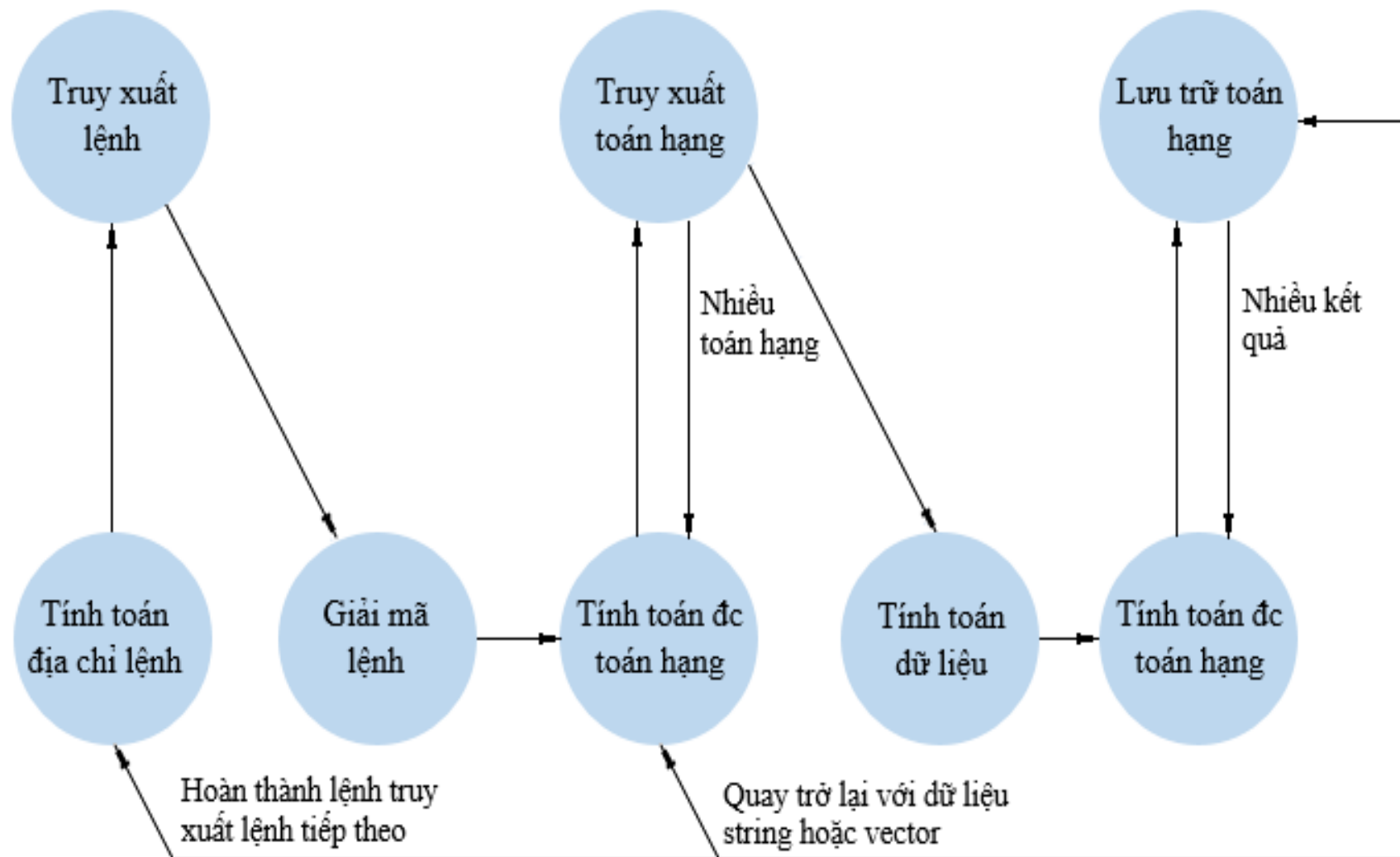
IR = Thanh ghi lệnh bộ nhớ

PC = Bộ đếm chương trình

Luồng dữ liệu chu kỳ ngắn



+ Bài 3. Luồng xử lý (Pipelining) lệnh



Sơ đồ trạng thái chu kỳ lệnh

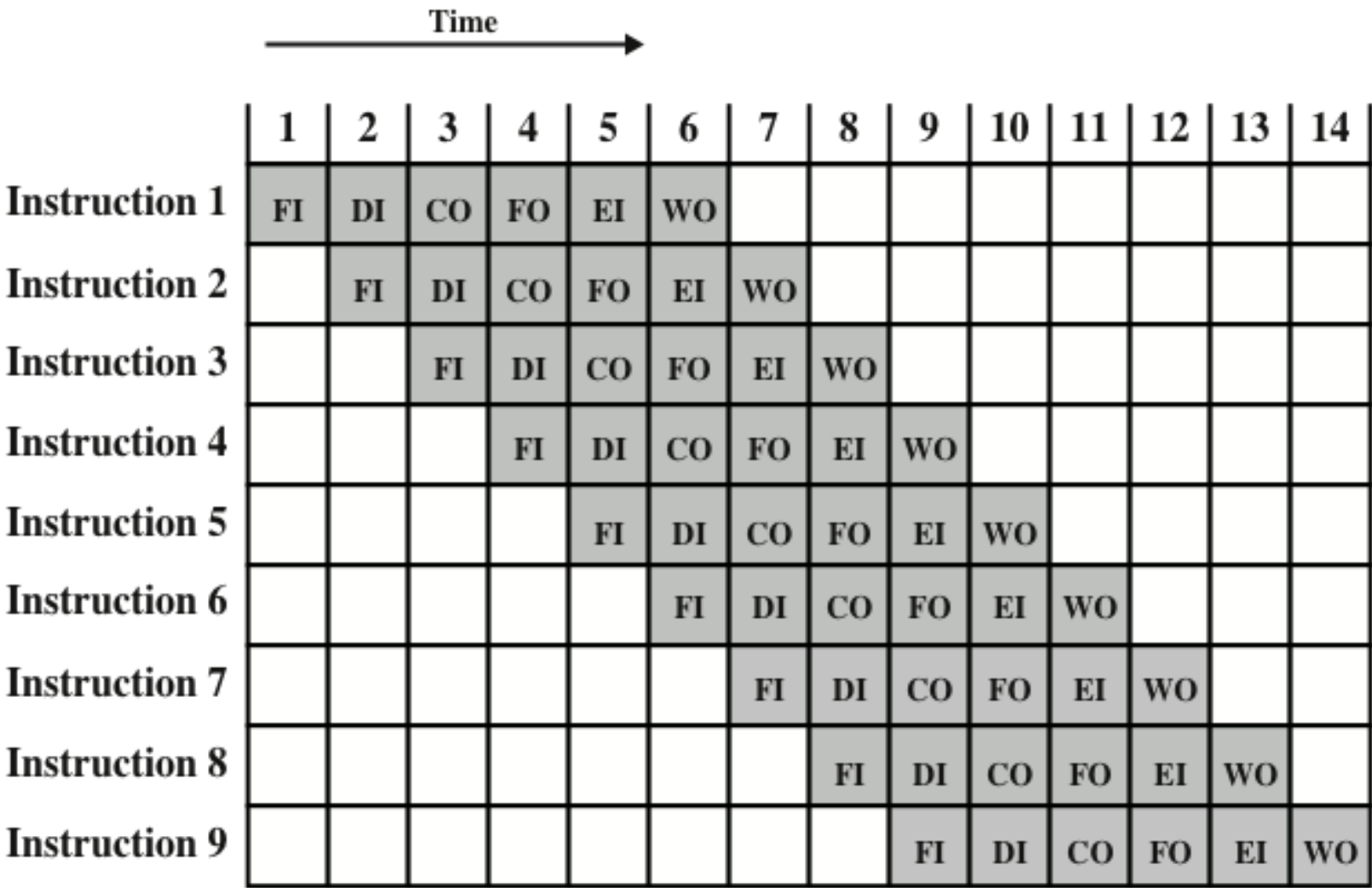
Bài 3. Luồng xử lý (Pipelining) lệnh

❖ Chu kỳ lệnh được chia thành 6 giai đoạn:

- Truy xuất lệnh (**FI** – Fetch instruction): Đọc lệnh tiếp theo vào bộ đệm.
- Giải mã lệnh (**DI** – Decode instruction): Giải mã opcode và nhận diện toán hạng.
- Tính toán địa chỉ toán hạng (**CO** – Calculate operands): Tính toán địa chỉ hiệu dụng của từng toán hạng nguồn: địa chỉ dịch chuyển, gián tiếp thanh ghi, gián tiếp .v.v....
- Truy xuất toán hạng (**FO** – Fetch operands): Truy xuất từng toán hạng từ bộ nhớ. Không cần truy xuất toán hạng từ thanh ghi
- Thực thi lệnh (**EI** – Execute instruction): Thực hiện hành động và lưu trữ kết quả (nếu có) trong vị trí toán hạng đích đã định.
- Ghi toán hạng (**WO** – Write operand): Lưu kết quả vào bộ nhớ.



Biểu đồ thời gian của pipeline lệnh



Ví dụ: kỹ thuật đường ống lệnh trong trường hợp câu lệnh rẽ nhánh



	Time →							← Branch Penalty						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO							
Instruction 5					FI	DI	CO							
Instruction 6						FI	DI							
Instruction 7							FI							
Instruction 15								FI	DI	CO	FO	EI	WO	
Instruction 16									FI	DI	CO	FO	EI	WO

V
ón
X
a=
b=

Mô tả khác về Pipeline

Time ↓

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

(a) No branches

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15					I3
9	I16	I15				
10		I16	I15			
11			I16	I15		
12				I16	I15	
13					I16	I15
14						I16

(b) With conditional branch

Xung đột trong kỹ thuật đường ống (Pipeline Hazard)

- Trong một số trường hợp, kỹ thuật đường ống bị đình trệ do một số xung đột như sau:
 - Xung đột tài nguyên: do nhiều công đoạn dùng chung một tài nguyên.
 - Xung đột dữ liệu: lệnh sau sử dụng kết quả của lệnh trước (một bộ phận phần cứng được dùng để đưa kết quả từ đầu ra ALU trực tiếp vào một trong các thanh ghi đầu vào)
 - Xung đột điều khiển: do rẽ nhánh gây ra (đóng băng kỹ thuật ống dẫn trong một chu kỳ)

Xung đột tài nguyên

- Hazard tài nguyên xảy ra khi hai hoặc nhiều lệnh đã ở trong đường ống cần dùng cùng một tài nguyên

	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO			
	I2		FI	DI	FO	EI	WO		
	I3			FI	DI	FO	EI	WO	
	I4				FI	DI	FO	EI	WO

(a) Five-stage pipeline, ideal case

- VD: Lệnh I1 truy xuất toán hạng từ bộ nhớ → xung đột với việc truy xuất lệnh I3 → việc truy xuất lệnh I3 phải chậm lại 1 chu kỳ (hình b)

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Idle	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) I1 source operand in memory

- Còn được gọi là Hazard cấu trúc

	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX	FI	DI	FO	EI	WO					
SUB ECX, EAX		FI	DI	Idle		FO	EI	WO		
I3			FI			DI	FO	EI	WO	
I4						FI	DI	FO	EI	WO

RAW

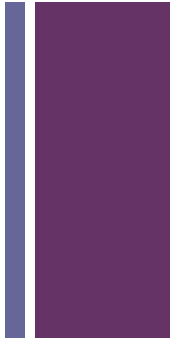
Hazard

+

Xung đột dữ liệu

Câu lệnh I2 sử dụng kết quả của câu lệnh I1 (EAX): việc truy xuất toán hạng phải chậm lại 2 chu kỳ đồng hồ để đợi câu lệnh I1 thực hiện xong việc ghi toán hạng

+ Các loại Hazard dữ liệu



❖ Đọc sau khi ghi (RAW)

- ❖ Một lệnh sửa đổi một thanh ghi hoặc vị trí bộ nhớ
- ❖ Lệnh tiếp theo đọc dữ liệu trong bộ nhớ hoặc vị trí thanh ghi
- ❖ Hazard xảy ra nếu việc đọc diễn ra trước khi hoạt động ghi hoàn tất

❖ Ghi sau khi đọc (WAR)

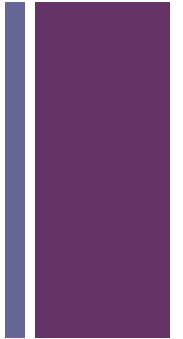
- ❖ Một lệnh đọc một thanh ghi hoặc vị trí bộ nhớ
- ❖ Lệnh tiếp theo ghi vào vị trí đó
- ❖ Hazard xảy ra nếu thao tác ghi hoàn thành trước khi có thao tác đọc

❖ Ghi sau khi ghi (WAW)

- ❖ Hai lệnh cùng ghi vào 1 vị trí
- ❖ Hazard xảy ra nếu các thao tác ghi diễn ra theo thứ tự ngược với trình tự dự định

+ Xung đột điều khiển

- ❖ Còn được gọi là Hazard rẽ nhánh
- ❖ Xảy ra khi kỹ thuật đường ống đưa ra dự báo nhánh bị sai so với nhánh thực tế
- ❖ Các lệnh được truy xuất sẽ bị loại bỏ.
- ❖ Các biện pháp đối phó với Hazard rẽ nhánh
 - Sử dụng nhiều luồng
 - Truy xuất trước mục tiêu rẽ nhánh
 - Bộ đệm vòng lặp
 - Dự báo rẽ nhánh
 - Rẽ nhánh chậm





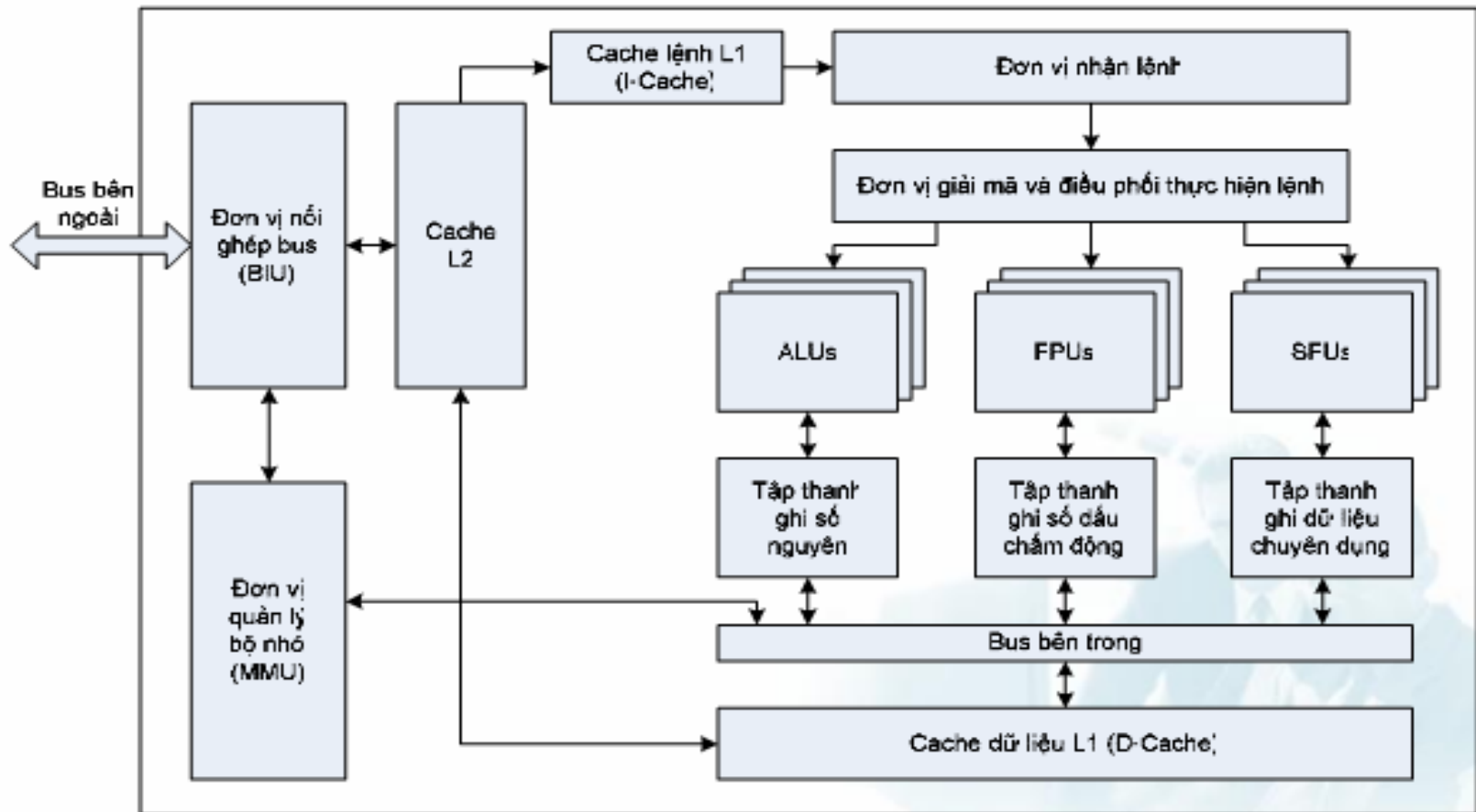
Bài 4. Kiến trúc VXL tiên tiến (đọc thêm)

26/8

6

1. Cấu trúc chung của các bộ xử lý tiên tiến
2. Các kiến trúc song song mức lệnh
3. Kiến trúc RISC

1. Cấu trúc chung của các BXL tiên tiến



Các đơn vị xử lý dữ liệu

- ❖ Các đơn vị số nguyên (ALU - Integer Unit – IU)
- ❖ Các đơn vị số dấu chấm động (Floating Point Unit – FPU)
- ❖ Các đơn vị chức năng đặc biệt (SFU):
 - ❖ Đơn vị xử lý dữ liệu âm thanh
 - ❖ Đơn vị xử lý dữ liệu hình ảnh
 - ❖ Đơn vị xử lý dữ liệu vector

Bộ nhớ cache

- Được tích hợp trên chip vi xử lý
- Thường bao gồm 2 mức Cache:
 - Cache L1 gồm 2 phần tách rời:
 - Cache lệnh
 - Cache dữ liệu
 - Giải quyết xung đột khi nhận lệnh và dữ liệu
 - Cache L2: dùng chung cho lệnh và dữ liệu

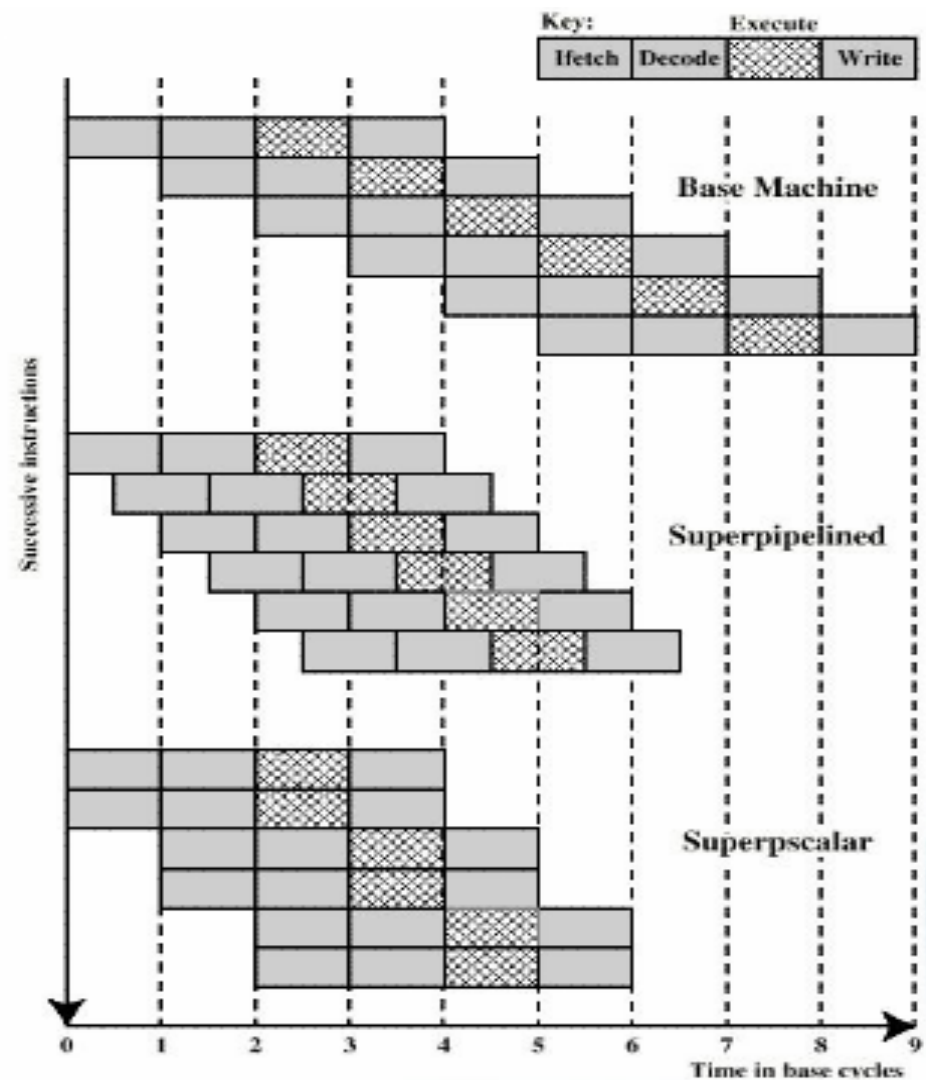
2. Kiến trúc song song mức lệnh

- ❖ Siêu đường ống (Superpipeline và Hyperpipeline)
- ❖ Siêu vô hướng (Superscalar)
- ❖ Từ lệnh dài – VLIW (Very Long Instruction Word)

Đơn vị quản lý bộ nhớ

- ❖ Chuyển đổi địa chỉ ảo thành địa chỉ vật lý
- ❖ Cung cấp cơ chế phân trang hoặc phân đoạn
- ❖ Cung cấp chế độ bảo vệ bộ nhớ

Siêu đường ống và siêu vô hướng

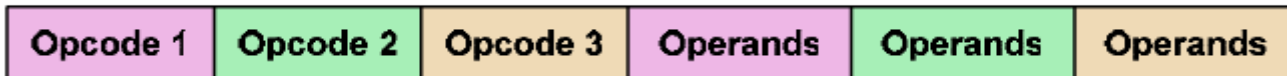


VLIW (Very Long Instruction Word)

Từ lệnh thông thường



Từ lệnh dài



3. Kiến trúc RISC

- CISC và RISC:

- CISC – Complex Instruction Set Computer:

- Máy tính có tập lệnh phức tạp

- VD: các bộ xử lý 80x86 ...

- RISC – Reduced Instruction Set Computer:

- Máy tính có tập lệnh rút gọn

- VD: các bộ xử lý Sun SPARC, Power PC, ...

Các đặc trưng của RISC

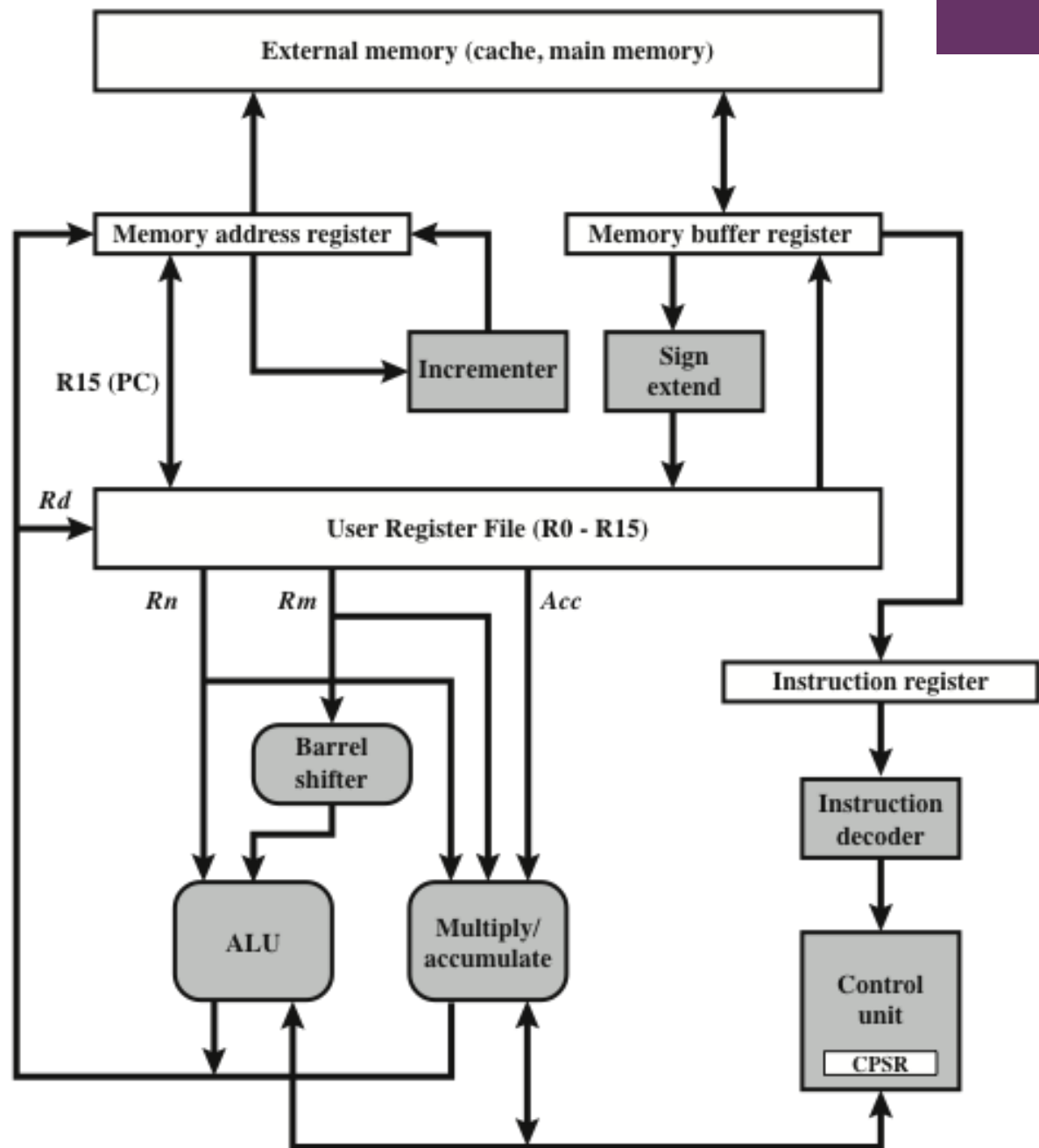
- ❖ Số lượng lệnh ít
- ❖ Các lệnh có thời gian thực hiện là 1 chu kỳ máy
- ❖ Độ dài của các lệnh bằng nhau (32 bit)
- ❖ Có ít khuôn dạng lệnh (≤ 4)
- ❖ Có ít chế độ địa chỉ hóa toán hạng (≤ 4)
- ❖ Có nhiều thanh ghi
- ❖ Các lệnh chủ yếu là thao tác giữa thanh ghi với thanh ghi
- ❖ Truy cập bộ nhớ thông qua 2 lệnh LOAD và STORE

+ Vi xử lý ARM

ARM cơ bản là hệ thống RISC với các đặc điểm sau:

- ❖ Moderate array of uniform registers
- ❖ Mô hình tải / lưu trữ của quá trình xử lý dữ liệu trong đó các phép toán chỉ thực hiện trên toán hạng trong thanh ghi mà không thực hiện trực tiếp trong bộ nhớ
- ❖ Lệnh có độ dài cố định 32 bit cho tập lệnh tiêu chuẩn và 16 bit cho tập lệnh Thumb
- ❖ ALU và shifter riêng biệt
- ❖ Có ít chế độ định địa chỉ. Tất cả các địa chỉ tải / lưu trữ được xác định từ thanh ghi và trường lệnh
- ❖ Các chế độ định địa chỉ tự động tăng và tự động giảm được sử dụng để cải thiện hoạt động của vòng lặp của chương trình
- ❖ Việc thực hiện lệnh có điều kiện giúp giảm thiểu nhu cầu lệnh rẽ nhánh điều kiện, do đó nâng cao hiệu quả pipeline

Tổ chức ARM



Chế độ bộ xử lý

- ❖ Kiến trúc ARM hỗ trợ bảy chế độ thực hiện
- ❖ Hầu hết chương trình ứng dụng thực thi trong chế độ người dùng
 - ❖ Trong khi bộ xử lý đang ở chế độ người dùng, chương trình đang được thực thi không thể truy cập vào tài nguyên hệ thống được bảo vệ hoặc thay đổi chế độ, chỉ có thể tạo ra một exception
- ❖ Sáu chế độ thực hiện còn lại được coi là chế độ ưu đãi (privileged mode)
 - ❖ Các chế độ này được sử dụng để chạy phần mềm hệ thống
- ❖ Thuận lợi khi có nhiều chế độ privileged khác nhau
 - ❖ Hệ điều hành có thể điều chỉnh việc sử dụng phần mềm hệ thống cho nhiều tình huống
 - ❖ Một số thanh ghi được dành riêng để dùng trong mỗi chế độ privileged, cho phép thay đổi nhanh hơn trong

Chế độ Exception

- ❖ Có quyền truy cập đầy đủ vào tài nguyên hệ thống và có thể thoải mái thay đổi chế độ
- ❖ Khi xảy ra các exception
- ❖ Các chế độ Exception:
 - ❖ Supervisor mode – giám sát
 - ❖ Abort mode – huỷ bỏ
 - ❖ Undefined mode – không xác định
 - ❖ Fast interrupt mode – gián đoạn nhanh
 - ❖ Interrupt mode – gián đoạn
- ❖ Chế độ hệ thống:
 - ❖ Không vào được bởi bất kỳ exception nào và sử dụng cùng các thanh ghi có sẵn trong Chế độ người dùng
 - ❖ Được dùng để chạy một số tác vụ hệ điều hành privileged
 - ❖ Có thể bị gián đoạn bởi một trong năm loại chế độ Exception



Tổ chức thanh ghi ARM

Modes						
		Privileged modes				
		Exception modes				
User	System	Supervisor	Abort	Undefined	Interrupt	Fast Interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13 (SP)	R13 (SP)	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14 (LR)	R14 (LR)	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Shading indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode.

SP = stack pointer
LR = link register
PC = program counter

CPSR = current program status register
SPSR = saved program status register



Format of ARM CPSR and SPSR

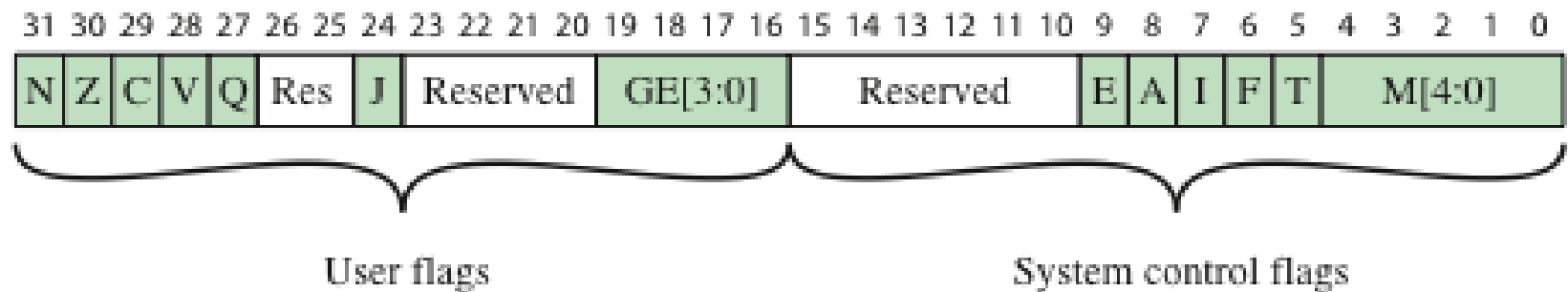


Figure 14.27 Format of ARM CPSR AND SPSR



Table 14.4

ARM Interrupt Vector

Exception type	Mode	Normal entry address	Description
Reset	Supervisor	0x00000000	Occurs when the system is initialized.
Data abort	Abort	0x00000010	Occurs when an invalid memory address has been accessed, such as if there is no physical memory for an address or the correct access permission is lacking.
FIQ (fast interrupt)	FIQ	0x0000001C	Occurs when an external device asserts the FIQ pin on the processor. An interrupt cannot be interrupted except by an FIQ. FIQ is designed to support a data transfer or channel process, and has sufficient private registers to remove the need for register saving in such applications, therefore minimizing the overhead of context switching. A fast interrupt cannot be interrupted.
IRQ (interrupt)	IRQ	0x00000018	Occurs when an external device asserts the IRQ pin on the processor. An interrupt cannot be interrupted except by an FIQ.
Prefetch abort	Abort	0x0000000C	Occurs when an attempt to fetch an instruction results in a memory fault. The exception is raised when the instruction enters the execute stage of the pipeline.
Undefined instructions	Undefined	0x00000004	Occurs when an instruction not in the instruction set reaches the execute stage of the pipeline.
Software interrupt	Supervisor	0x00000008	Generally used to allow user mode programs to call the OS. The user program executes a SWI instruction with an argument that identifies the function the user wishes to perform.



Tổng kết chương

- Tổ chức bộ xử lý
- Tổ chức thanh ghi
 - Thanh ghi hiển thị người dùng
 - Thanh ghi điều khiển và trạng thái
- Chu kỳ lệnh
 - Chu kỳ gián tiếp
 - Luồng dữ liệu
- Họ vi xử lý x86
 - Tổ chức thanh ghi
 - Xử lý gián đoạn

Cấu trúc và Chức năng Bộ xử lý

- Pipelining lệnh
 - Chiến lược Pipelining
 - Hiệu suất Pipeline
 - Pipeline hazard
 - Xử lý rẽ nhánh
 - Pipelining Intel 80486
- Bộ xử lý ARM
 - Tổ chức bộ xử lý
 - Chế độ bộ xử lý
 - Tổ chức thanh ghi
 - Xử lý gián đoạn

12.4 Kỹ thuật đường ống Pipelining

- Chu kỳ lệnh được chia thành 6 giai đoạn:
 - Truy xuất lệnh (FI – Fetch instruction): Đọc lệnh tiếp theo vào bộ đệm.
 - Giải mã lệnh (DI – Decode instruction): Giải mã opcode và nhận diện toán hạng.
 - Tính toán địa chỉ toán hạng (CO – Calculate operands): Tính toán địa chỉ hiệu dụng của từng toán hạng nguồn: địa chỉ dịch chuyển, gián tiếp thanh ghi, gián tiếp .v.v....
 - Truy xuất toán hạng (FO – Fetch operands): Truy xuất từng toán hạng từ bộ nhớ. Không cần truy xuất toán hạng từ thanh ghi
 - Thực thi lệnh (EI – Execute instruction): Thực hiện hành động và lưu trữ kết quả (nếu có) trong vị trí toán hạng đích đã định.
 - Ghi toán hạng (WO – Write operand): Lưu kết quả vào bộ nhớ.

Biểu đồ thời gian của pipeline lệnh

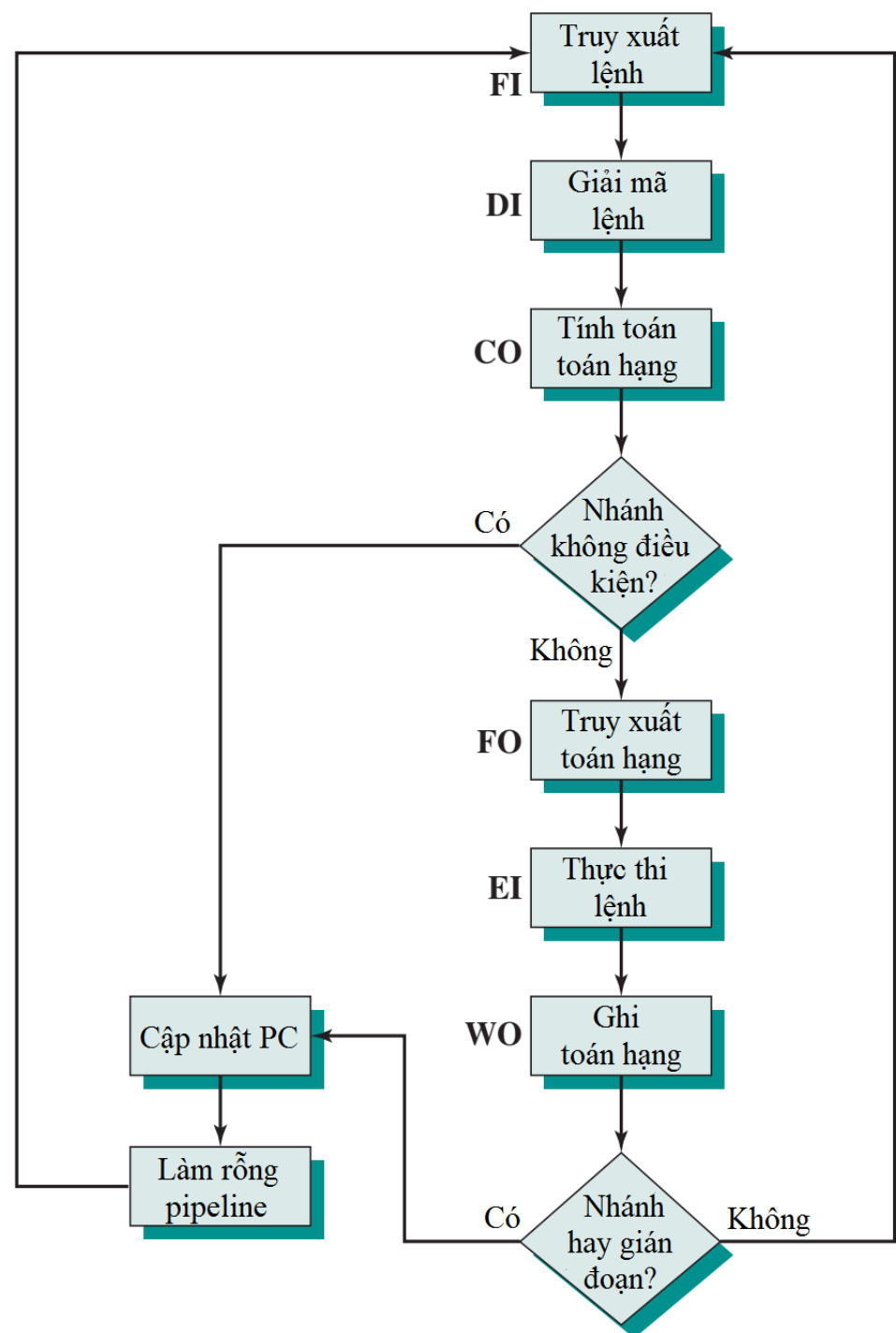
	Time →													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Ví dụ: kỹ thuật đường ống lệnh trong trường hợp câu lệnh rẽ nhánh

	Time →							← Branch Penalty						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO							
Instruction 5					FI	DI	CO							
Instruction 6						FI	DI							
Instruction 7							FI							
Instruction 15								FI	DI	CO	FO	EI	WO	
Instruction 16									FI	DI	CO	FO	EI	WO



Pipeline lệnh 6 giai đoạn



Mô tả khác về Pipeline

Time ↓

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

(a) No branches

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15					I3
9	I16	I15				
10		I16	I15			
11			I16	I15		
12				I16	I15	
13					I16	I15
14						I16

(b) With conditional branch

Xung đột trong kỹ thuật đường ống (Pipeline Hazard)

- Trong một số trường hợp, kỹ thuật đường ống bị đình trệ do một số xung đột như sau:
 - Xung đột tài nguyên: do nhiều công đoạn dùng chung một tài nguyên.
 - Xung đột dữ liệu: lệnh sau sử dụng kết quả của lệnh trước (một bộ phận phần cứng được dùng để đưa kết quả từ đầu ra ALU trực tiếp vào một trong các thanh ghi đầu vào)
 - Xung đột điều khiển: do rẽ nhánh gây ra (đóng băng kỹ thuật ống dẫn trong một chu kỳ)



Xung đột tài nguyên

- Hazard tài nguyên xảy ra khi hai hoặc nhiều lệnh đã ở trong đường ống cần dùng cùng một tài nguyên
- VD: Lệnh I1 truy xuất toán hạng từ bộ nhớ → xung đột với việc truy xuất lệnh I3 → việc truy xuất lệnh I3 phải chậm lại 1 chu kỳ (hình b)
- Còn được gọi là Hazard cấu trúc

	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO			
	I2		FI	DI	FO	EI	WO		
	I3			FI	DI	FO	EI	WO	
	I4				FI	DI	FO	EI	WO

(a) Five-stage pipeline, ideal case

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Idle	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) I1 source operand in memory

	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX	FI	DI	FO	EI	WO					
SUB ECX, EAX		FI	DI	Idle		FO	EI	WO		
I3			FI			DI	FO	EI	WO	
I4						FI	DI	FO	EI	WO

RAW

Hazard

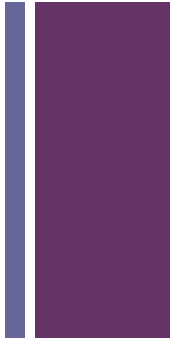
+

Xung đột dữ liệu

Câu lệnh I2 sử dụng kết quả của câu lệnh I1 (EAX): việc truy xuất toán hạng phải chậm lại 2 chu kỳ đồng hồ để đợi câu lệnh I1 thực hiện xong việc ghi toán hạng

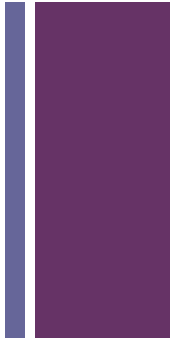


Các loại Hazard dữ liệu



- **Đọc sau khi ghi (RAW)**
 - Một lệnh sửa đổi một thanh ghi hoặc vị trí bộ nhớ
 - Lệnh tiếp theo đọc dữ liệu trong bộ nhớ hoặc vị trí thanh ghi
 - Hazard xảy ra nếu việc đọc diễn ra trước khi hoạt động ghi hoàn tất
- **Ghi sau khi đọc (WAR)**
 - Một lệnh đọc một thanh ghi hoặc vị trí bộ nhớ
 - Lệnh tiếp theo ghi vào vị trí đó
 - Hazard xảy ra nếu thao tác ghi hoàn thành trước khi có thao tác đọc
- **Ghi sau khi ghi (WAW)**
 - Hai lệnh cùng ghi vào 1 vị trí
 - Hazard xảy ra nếu các thao tác ghi diễn ra theo thứ tự ngược với trình tự dự định

+ Xung đột điều khiển



- Còn được gọi là Hazard nhánh
- Xảy ra khi kỹ thuật đường ống đưa ra dự báo nhánh bị sai so với nhánh thực tế
- Các lệnh được truy xuất sẽ bị loại bỏ

+ 12.5 Kiến trúc VXL tiên tiến

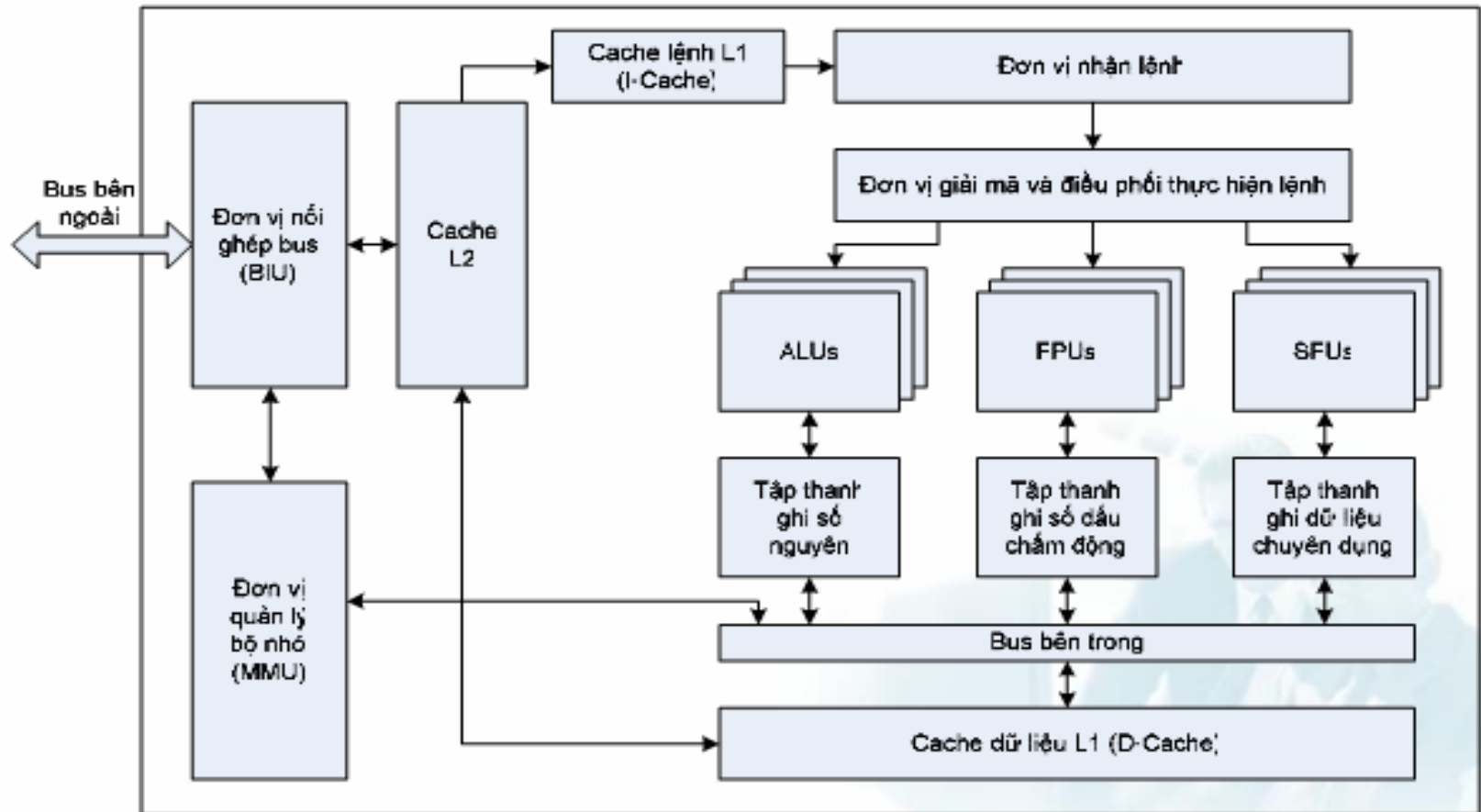
54/8

6

- a. Cấu trúc chung của các bộ xử lý tiên tiến
- b. Các kiến trúc song song mức lệnh
- c. Kiến trúc RISC

12.5 Kiến trúc VXL tiên tiến

a. Cấu trúc chung của các BXL tiên tiến



12.5 Kiến trúc VXL tiên tiến

Các đơn vị xử lý dữ liệu

- Các đơn vị số nguyên (Integer Unit – IU)
- Các đơn vị số dấu chấm động (Floating Point Unit – FPU)
- Các đơn vị chức năng đặc biệt:
 - Đơn vị xử lý dữ liệu âm thanh
 - Đơn vị xử lý dữ liệu hình ảnh
 - Đơn vị xử lý dữ liệu vector

12.5 Kiến trúc VXL tiên tiến

Bộ nhớ cache

- Được tích hợp trên chip vi xử lý
- Thường bao gồm 2 mức Cache:
 - Cache L1 gồm 2 phần tách rời:
 - Cache lệnh
 - Cache dữ liệu
 - Giải quyết xung đột khi nhận lệnh và dữ liệu
 - Cache L2: dùng chung cho lệnh và dữ liệu

12.5 Kiến trúc VXL tiên tiến

Đơn vị quản lý bộ nhớ

- Chuyển đổi địa chỉ ảo thành địa chỉ vật lý
- Cung cấp cơ chế phân trang hoặc phân đoạn
- Cung cấp chế độ bảo vệ bộ nhớ